CS 499 Milestone Three Narrative

Enhancement Two: Algorithms and Data Structures by John Grudzinski

Brief Description of the Artifact

The artifact I selected for the algorithms and data structures category is my IT 145 Grazioso Salvare Rescue Animal System. The original program allowed a user to intake new dogs and monkeys, reserve an animal, print all dogs, print all monkeys, and print available animals. The way it works is by using object-oriented programming through a parent RescueAnimal class and two child classes, Dog and Monkey. The artifact was a good foundation because it already included inheritance, menu driven control flow, validation, lists, searching, and filtering.

Justification for Inclusion in the ePortfolio

I selected this artifact because it gave me a clear opportunity to showcase my growth in algorithms and data structures. The original version worked for what the initial course required, but it relied heavily on separate ArrayList collections for dogs and monkeys. There were also duplicate checks and reservation searches which required linear scans through those lists, and similar logic was repeated in multiple places. I figured for a small class project, that approach was acceptable, but it did not scale well and did not fully show the data structure decision making that I want to demonstrate in my portfolio.

The enhanced version I created improves the artifact by adding a new AnimalInventory class that centralizes animal storage, lookup, filtering, reservation, and sorting. This class uses a *HashMap<String, RescueAnimal>* for fast lookup by a combined animal type and name key, such as *dog:spot* or *monkey:george*. This changes direct duplicate checks from linear time, O(n), to average constant time, O(1). I also kept a list of RescueAnimal objects because lists are still useful for filtering and sorted output. I believe this combination demonstrates an important

design trade off where a HashMap is better for direct lookup, while a list is better for producing ordered reports.

I also improved the use of inheritance by setting the animal type in the Dog and Monkey constructors and managing both animal types through the shared RescueAnimal parent class where possible. The enhanced Driver class now relies on the AnimalInventory class instead of directly managing separate dog and monkey lists. This reduces repeated code and makes the application easier to extend if Grazioso Salvare wanted to add more rescue animal types later on. Some additional algorithmic improvements I added include using *LinkedHashSet* collections for valid monkey species and training statuses, since those values are mainly used for membership checks. I also added reusable validation helpers for common input patterns such as animal type, gender, numeric ranges, decimal values, Boolean values, country names, acquisition dates, training statuses, and monkey species. Finally, I added sorted output using a Comparator so animals are displayed consistently by type, country, and name. These improvements make the program more organized, predictable, and maintainable.

Course Outcome Alignment

This enhancement most directly supports Course Outcome 3 for designing and evaluating computing solutions using algorithmic principles and computer science practices while managing trade-offs involved in design choices. The way I met this outcome is by evaluating the limitations of repeated ArrayList searches and improving lookup behavior with a HashMap. I also used a list for sorted output, which shows that I considered the trade-offs between lookup efficiency and reporting needs rather than assuming one data structure should handle every task. This enhancement also supports Course Outcome 4 because I used well founded programming techniques to make the application more useful and maintainable. The new AnimalInventory

class separates collection management from user-interface/menu logic, and the reusable validation helpers reduce duplicate code in *Driver.java*. The working product now better demonstrates object-oriented design, collection management, filtering, sorting, and maintainability.

The enhancement partially supports Course Outcome 2 because I documented the artifact and can explain the algorithmic decisions clearly through this narrative and my ePortfolio. It also makes some progress toward Course Outcome 5 because the improved input validation and centralized logic reduce structural and logic flaws that could lead to inconsistent or invalid records. I still plan to demonstrate Course Outcome 1 more strongly in the database dashboard enhancement, where the focus will be on data driven decision making.

Reflection on the Enhancement Process

Enhancing this artifact helped me better understand that algorithms and data structures are not just abstract topics from class because they affect how maintainable and scalable an application feels in practice. The original application worked, but the repeated loops and separate lists made it harder to reason about the program as the amount of data grew. Creating the AnimalInventory class helped me separate concerns and made the program easier to follow because *Driver.java* now focuses more on user interaction, while AnimalInventory handles storage, lookup, filtering, reservation, and sorting.

One challenge was deciding how far to change the original project without losing the purpose of the assignment. I wanted the enhancement to be significant, but I also wanted the application to remain recognizable as the original IT 145 artifact. I addressed this by keeping the original console based menu and the existing RescueAnimal, Dog, and Monkey structure while

improving the underlying data handling. This allowed the artifact to remain close and feel connected to the original coursework while still showing meaningful growth in my skillset. Another challenge I faced was choosing the right data structures for different tasks. A HashMap improved direct lookup and duplicate checking, but it was not the best structure for sorted display. I handled this by using both a HashMap and a list. The HashMap supports efficient lookup, while the list supports filtering and Comparator based sorting. This reinforced the idea that good software design often involves combining tools and structures based on their strengths rather than forcing one structure to solve every problem.

Overall, this enhancement improved the artifact by making the application more efficient, better organized, and easier to extend. It demonstrates my ability to identify limitations in existing code, apply data structures intentionally, reduce repeated logic, and explain the trade-offs behind my design choices. These are important skills for the specialization I chose which is secure full-stack software development because professional developers often need to improve existing systems while preserving functionality and making the code easier to maintain over time.